

Project Report: RideOS

Course: OS

Instructor: Sir Zulfiqar Ali

Group Members:

Ali Kashif	24K-0802	k240802@nu.edu.pk
Ismail Silat	24K-0546	k240546@nu.edu.pk
Hammad Abdul Rahim	24K-0581	k240581@nu.edu.pk

Language & Tools

- **Language:** C
- **Graphics Library:** Raylib (for GUI visualization)
- **Development Environment:** Cross-platform (Linux (Ubuntu) & MacOS)

Project Overview

RideOS is a high-performance, multithreaded autonomous dispatch simulation system designed to model the complexities of real-time ride-sharing logistics. The system focuses on demonstrating advanced Operating Systems concepts such as concurrent process management, inter-process communication (IPC), and priority-based resource scheduling.

The application utilizes a dual-process architecture where a backend "Main Server" handles the heavy lifting of request processing and driver dispatching, while a frontend "Request Server" provides a real-time graphical dashboard. The two processes remain synchronized using POSIX Shared Memory and Semaphores, while ride requests are transmitted through high-speed Named Pipes (FIFOs).

Key Features:

- **Dual-Process Architecture:** Decoupled backend (Main Server) and frontend (GUI) for modularity.
- **Multithreaded Dispatcher:** Uses a producer-consumer model to manage requests and drivers concurrently.
- **Priority-Based Scheduling:** Advanced queuing system for Normal, VIP, and Emergency requests.
- **Dynamic Aging Mechanism:** Prevents process starvation by promoting long-waiting requests to higher priority tiers.
- **Real-Time Interface:** A 60-FPS dashboard visualizing the state of the entire driver fleet.
- **IPC Synchronization:** Robust data integrity using Mutexes, Condition Variables, and Semaphores.

Objectives

Demonstrate IPC Implementation: Utilize Named Pipes for command passing and Shared Memory for high-frequency state updates.

Concurrent Resource Management: Implement a multithreaded pool of drivers and request handlers using Pthreads.

Priority Scheduling Simulation: Design an algorithm that balances system throughput with strict priority requirements (Emergency > VIP > Normal).

Prevent Resource Starvation: Apply the "Aging" concept where lower-priority requests gain priority over time to ensure they are eventually serviced.

Synchronization & Race Condition Prevention: Use Mutexes and Semaphores to protect critical sections and shared resources in a concurrent environment.

End-to-End Simulation Lifecycle: Manage a complete simulation flow from system configuration and initialization to real-time execution and final performance metrics reporting.

System Modules

Module 1: Backend Dispatch Engine (Main Server)

Description:

The core engine responsible for system state, driver management, and the central dispatching logic.

Core Features:

- Maintains state for up to 128 drivers (Online, Busy, Offline).
- Coordinates the matching of pending requests to available drivers based on priority.
- Spawns a dedicated thread for every active request to manage its lifecycle independently.
- Uses timed threads to simulate trip durations without blocking system logic.

IPC Handling:

Creates and manages the Named Pipe and Shared Memory.

Module 2: Graphical User Interface & Generator (Request Server)

Description:

The frontend application that handles user interaction and simulates the "client-side" of the ride-sharing platform.

Core Features:

- Visualizes driver cards with live status updates (color-coded).
- A background thread that simulates varying customer demand.
- Allows users to inject specific request types (Normal, VIP, Emergency) into the system for stress testing.
- Manages transitions between Intro, Config, Dashboard, and Metrics screens.

Visualization:

Implements state diffing to detect changes in shared memory and update the GUI activity feed dynamically.

Some Screenshots of GUI:

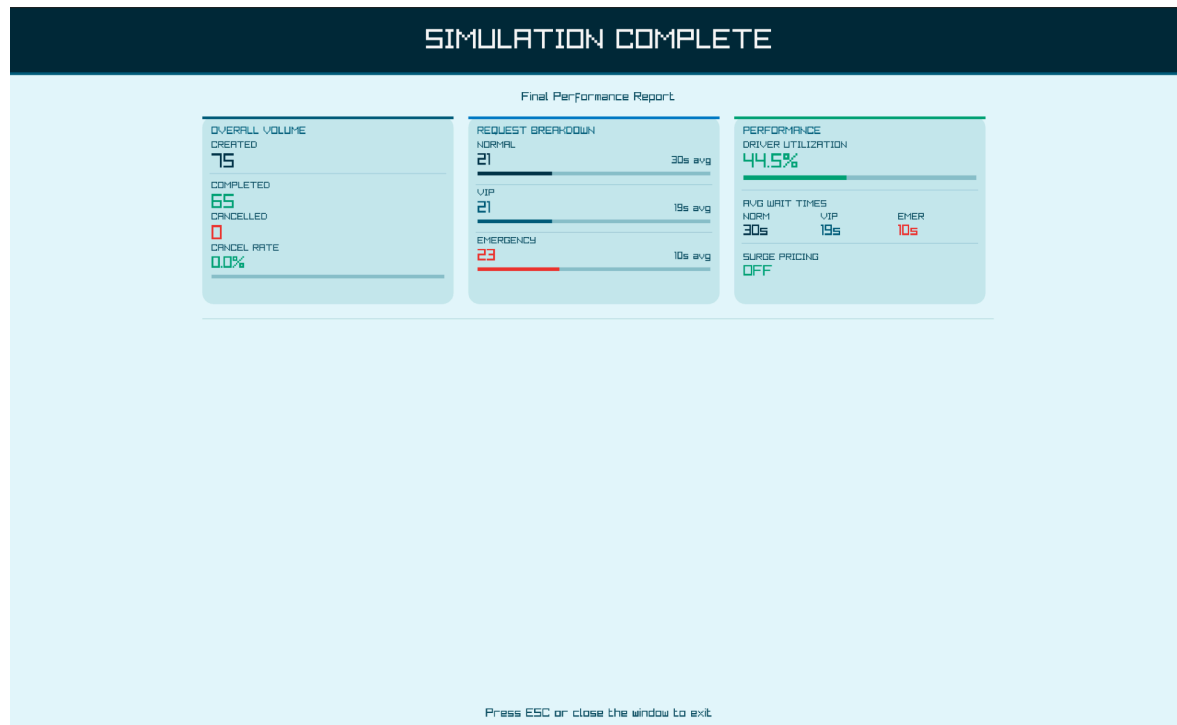
Intro Screen



Dashboard



Metrics Screen



Module 3: IPC & Synchronization Layer

Description:

The bridge between the two processes, ensuring thread-safe data transfer.

Core Features:

- Implements named pipes (FIFOs) for message passing for request submission.
- Uses Shared Memory for state synchronization for the GUI dashboard.
- Semaphores ensure mutual exclusion for the shared memory segment.
- Condition variables allow for efficient thread waking (Dispatcher waking Request threads) to avoid busy-waiting.

Module 4: Performance Analytics & Metrics

Description:

Records and analyzes system behavior to provide a post-simulation performance audit.

Core Features:

- Tracks timestamps for request creation, assignment, and completion.
- Calculates the percentage of time the driver fleet is active.
- Monitors cancellation of requests that exceeded their patience threshold (timeouts).

Expected Outcomes

- **High Concurrency:** Successful handling of dozens of simultaneous threads (drivers + requests) without deadlocks or race conditions.
- **Reliable IPC:** Seamless communication between the backend and frontend with zero message loss in the pipes.
- **Priority Compliance:** Verification that Emergency requests consistently achieve lower average wait times than standard requests.
- **Efficient GUI:** A responsive dashboard that reflects the internal state of the simulation at 60 FPS without impacting the backend dispatcher's performance.
- **System Stability:** Graceful shutdown and resource cleanup (unlinking pipes, shared memory, and semaphores) upon simulation end.

Code, How to Run, and File Structure

Github Repo for code: <https://github.com/ali5917/rideOS>

Prerequisites

- GCC Compiler (C11)
- Raylib 4.5+
- Linux environment or WSL (Required for POSIX IPC)

Installation Steps

1. Clone the repository.
2. Build the project: `make`
3. Launch the Main Server: `./main_server_bin`
4. Launch the Request Server: `./request_server_bin`

File Structure:

```
RideOS/
├── main_server/
│   ├── include/      # Backend headers (dispatch.h, request.h, etc.)
│   └── src/          # Dispatcher logic, queue management, and metrics
├── request_server/
│   ├── include/      # Frontend headers (gui.h, generator.h)
│   ├── src/          # Raylib dashboard, generator thread, and main loop
│   └── assets/        # UI PNGs and driver names list
├── shared/
│   └── include/       # IPC shared structures (PipeRequest, SharedState)
├── Makefile          # Unified build system
└── README.md         # Technical overview and controls
```